Database Management

Unit I

Outline

- Basic terms
- Database approach
- Concepts
- Architecture
- Database Design

Data

- The term data referred to facts concerning objects and events that could be recorded and stored on computer media
- Data can be structured and unstructured types
- For example, in a salesperson's database, the data would include facts such as customer name, address, and telephone number
- This type of data is called structured data
- The most important structured data types are numeric, character, and dates
- Structured data are stored in tabular form (in tables, relations, arrays, spreadsheets, etc.) and are most commonly found in traditional databases and data warehouses.

Database

Database

- Collection of related data

Ex : recording name , phone number and address of our known persons in a address book or digital diary is a database,

Database Management System (DBMS)

- DBMS is a collection of programs that enables users to create and maintain a database.

- It is a general-purpose software system that facilitates the process of
  - defining,
  - constructing,
  - manipulating and
  - sharing database

  among various users and applications

Database Management System (DBMS)

- Defining a database involves specifying the datatypes , structures and constraints of the data to be stored in the database. The database definition is also stored by the DBMS in the form of a database catalog (called meta-data)
- Constructing the database is the process of storing the data on some storage medium that is controlled by the DBMS
- Manipulating a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the mini-world, and generating report from the data
- Sharing a database allows multiple users and programs to access the database simultaneously

Database Management System (DBMS)

- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use
- Database Applications:
  - Banking: transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
- Databases can be very large.
- Databases touch all aspects of our lives

A simplified architecture for a database system

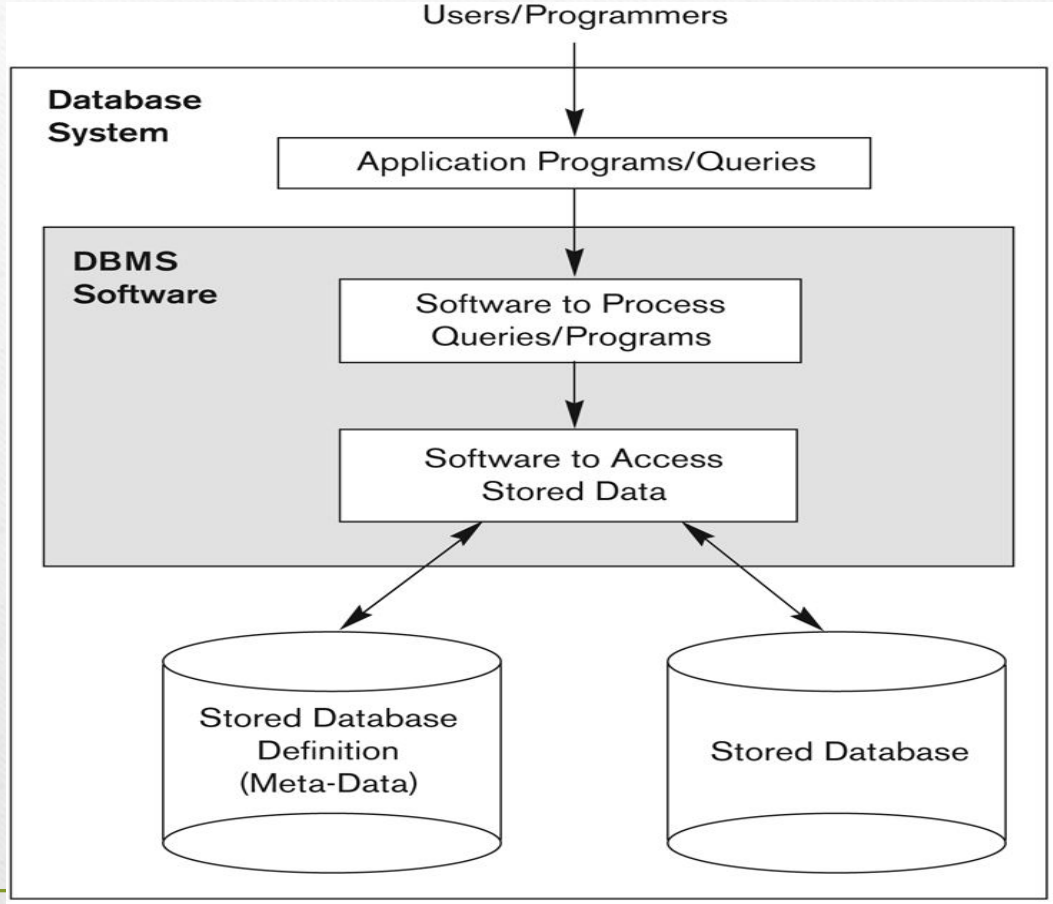


Figure 1.1
A simplified database system environment.

Example of a Database (with a Conceptual Data Model)

- **Mini-world for the example:**
  - Part of a UNIVERSITY environment
- **Some mini-world *entities*:**
  - STUDENTs
  - COURSEs
  - SECTIONs (of COURSEs)
  - (Academic) DEPARTMENTs
  - INSTRUCTORs

Example of a Simple Database

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

Example of a Database (with a Conceptual Data Model)

- **Some mini-world *relationships*:**
  - SECTIONs *are of specific* COURSEs
  - STUDENTs *take* SECTIONs
  - COURSEs *have prerequisite* COURSEs
  - INSTRUCTORs *teach* SECTIONs
  - COURSEs *are offered by* DEPARTMENTs
  - STUDENTs *major in* DEPARTMENTs
- Note: The above entities and relationships are typically expressed in a conceptual data model, such as the entity-relationship (ER) data or UML class model (see Chapters 3, 4)

Columns

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows

Example of a Simplified Database Catalog

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
….	….	…..
….	….	…..
….	….	…..
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerared type with all known majors. XXXXNNNN is used to define a type with four alpha characters followed by four digits

Figure 1.3
An example of a database catalog for the database in Figure 1.2.

File System vs DBMS

- In the early days, database applications were built directly on top of file systems

- Consider a bank enterprise as an example:
    - It keeps information about all customer and savings account
    - Incase if the information are stored in operating system files
    - To allow users to manipulate the information, the system has a number of application programs that manipulates the files, including programs to:

File System vs DBMS

- Debit or credit account
- Add a new account
- Find the balance of an account

Drawbacks of using file systems to store data

- Data redundancy and inconsistency
  - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation
  - Multiple files and formats
- Integrity problems
  - Integrity constraints (e.g., account balance > 0) become "buried" in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

Drawbacks of using file systems to store data (Cont.)

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
  - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems

View

- A database can be accessed by many users, and each of them may have a different perspective or view of the data.
- A database system provides a facility to define different views of the data for different users.
- A view is a subset of the database that contains virtual data derived from the database files but it does not in physical form.
- That is, no physical file is created for storing the data values of the view rather only the definition of the view is created

Schema & Instance

The database schema refers to the overall structure of the database, that is , all the information that is going to be represented in the database like name of the tables, data types of the each data item of each table, constraints, relationship between the tables, etc.

- **Instance** – the actual content of the database at a particular point in time
  - Analogous to the value of a variable

DBMS Architecture

- The main purpose of a DBMS is to provide users with an abstract view of the database
- The DBMS architecture describes how data in the database is viewed by the users

DBMS Architecture

- In this architecture, the overall database description can be defined at three levels, namely, **internal, conceptual , and external** levels and thus, named three-level DBMS architecture .
- This architecture is proposed by ANSI/SPARC (American National Standards Institute/Standards Planning and Requirements Committee) and hence, is also known as ANSI/SPARC architecture

DBMS Architecture

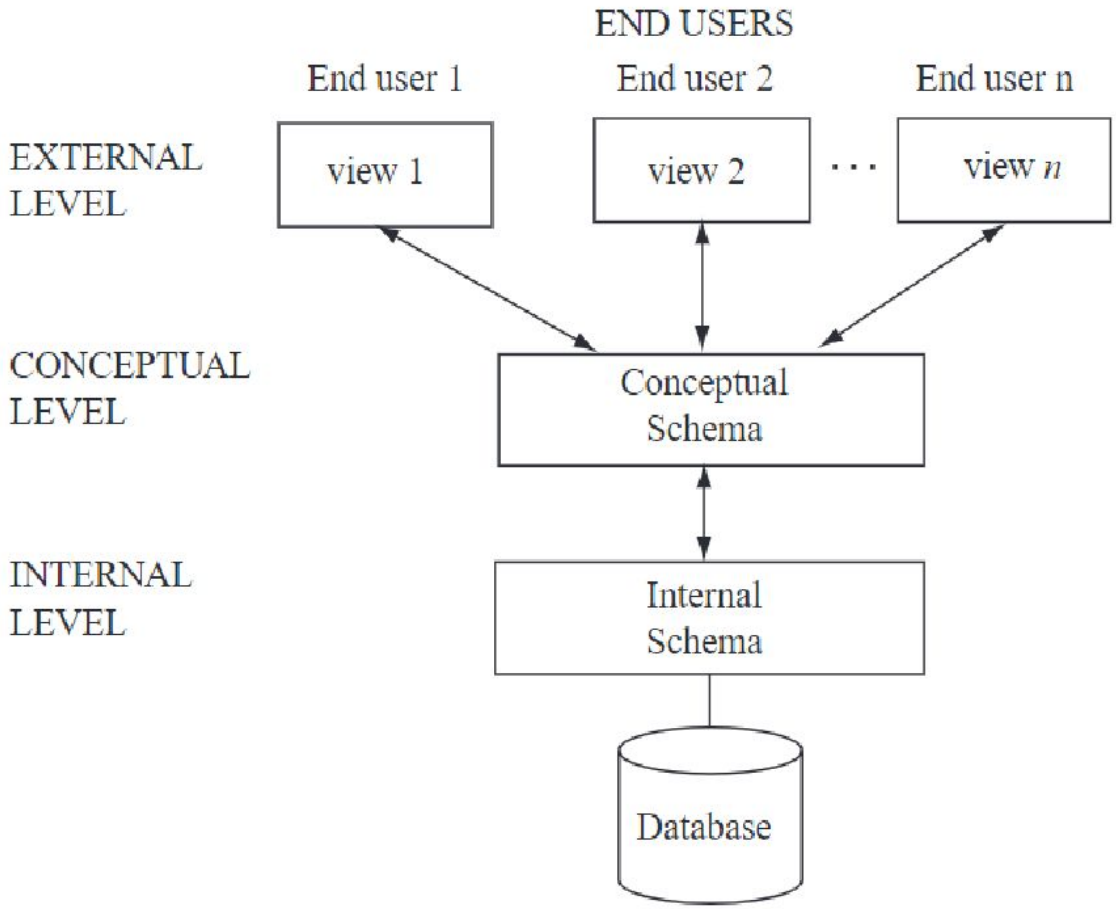


Fig. 1.1 *Three-schema architecture*

DBMS Architecture

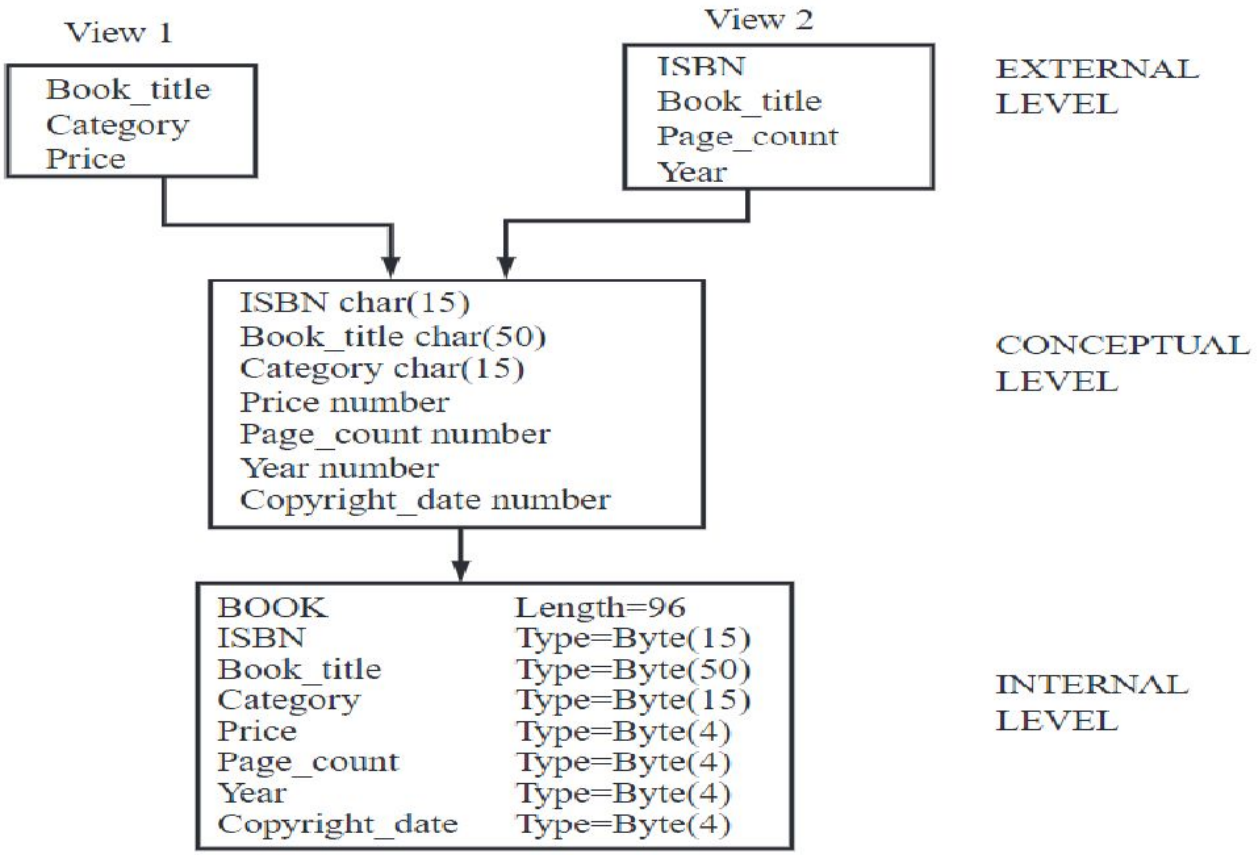


Fig. 1.2 *Three levels of Online Book database (BOOK file)*

Levels of Abstraction

- **Physical level:**
  - describes how a record (e.g., instructor) is stored – block of consecutive storage locations

- **Logical level:** describes data stored in database, and the relationships among the data.

  type *instructor* = **record**

  ID : string;
  name : string;
  dept_name : string;
  salary : integer;

  end;

- **View level:** set of application programs that hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

DBMS Architecture

Internal level (Physical level):

- It is the lowest level of data abstraction that deals with the physical representation of the database on the computer and thus, is also known as physical level .
- It describes how the data is physically stored and organized on the storage medium.
- At this level, various aspects are considered to achieve optimal runtime performance and storage space utilization.
- These aspects include storage space allocation techniques for data and indexes, access paths such as indexes, data compression and encryption techniques, and record placement

DBMS Architecture

Conceptual level (Logical level)

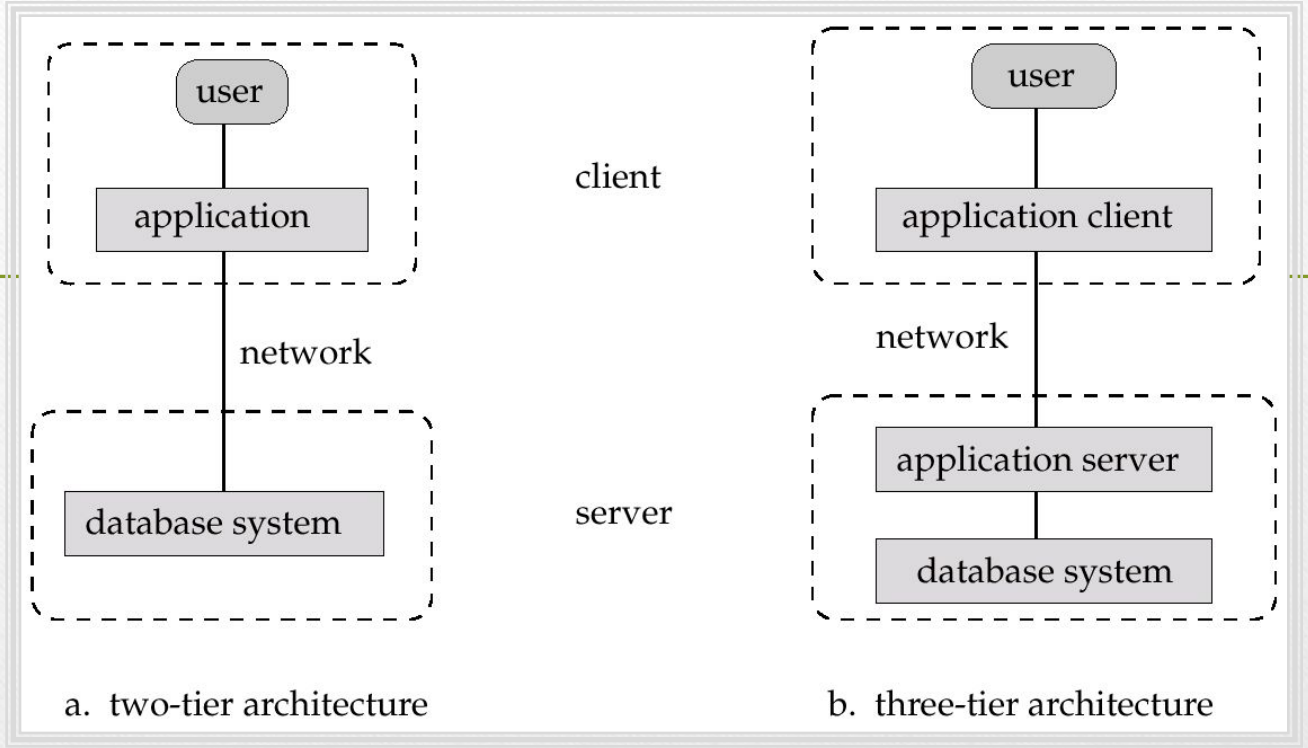
- This level of abstraction deals with the logical structure of the entire database and thus, is also known as logical level .
- It describes what data is stored in the database, the relationships among the data and complete view of the users requirements without any concern for the physical implementation.
- That is, it hides the complexity of physical storage structures.
- The conceptual view is the overall view of the database and it includes all the information that is going to be represented in the database

DBMS Architecture

External level (View level)

- It is the highest level of abstraction that deals with the users view of the database and thus, is also known as view level .
- In general, most of the users and application programs do not require the entire data stored in the database.
- The external level describes a part of the data-base for a particular group of users.
- It permits users to access data in a way that is customized according to their needs, so that the same data can be seen by different users in different ways, at the same time.
- In this way, it provides a powerful and security mechanism by hiding the parts of the database from certain users, as the user is not aware of existence of any attributes that are missing from the view.

Application Architectures



- **Two-tier architecture**: E.g. client programs using ODBC/JDBC to communicate with a database
- **Three-tier architecture**: E.g. web-based applications, and applications built using "middleware"

DBMS Architecture

Schema

These three levels are used to describe the schema of the database at various levels.

Thus, the three-level architecture is also known as three-schema architecture .

The internal level has an internal schema , which describes the physical storage structure of the database.

The conceptual level has a conceptual schema , which describes the structure of entire database.

The external level has external schemas or user views , which describe the part of the database according to a particular users requirements, and hide the rest of the database from that user.

DBMS Architecture

Whenever a user specifies a request to generate a new external view, the DBMS must transform the request specified at external level into a request at conceptual level, and then into a request at physical level.

If the user requests for data retrieval, the data extracted from the database must be presented according to the need of the user.

This process of transforming the requests and results between various levels of DBMS architecture is known as **mapping** .

The main advantage of three-schema architecture is that it provides **data independence**.

Data Independence

Data independence is the ability to change the schema at one level of the database system without having to change the schema at other levels.

Data independence is of two types, namely logical data independence and physical data independence

Data Independence

Logical data independence:

- It is the ability to change the conceptual schema without affecting the external schemas or application programs.
- The conceptual schema may be changed due to change in constraints or addition of new data item or removal of existing data item, etc., from the database.
- The separation of the external level from the conceptual level enables the users to make changes at the conceptual level without affecting the external level or the application programs.

Data Independence

Physical data independence:

- It is the ability to change the internal schema without affecting the conceptual or external schema.
- An internal schema may be changed due to several reasons such as for creating additional access structure, changing the storage structure, etc.
- The separation of internal schema from the conceptual schema facilitates physical data independence.